

Studio Video Editor — Implementation Guide for Developers

Audience: Frontend, backend, and media/render engineers

Version: 1.0

Status: Authoritative build spec for the click-to-edit video studio

This document explains **how to build** the Shyara Digital studio editor: UI behaviour, the shared animation engine, browser preview, and server-side MP4 export. It is not a general video editor — it is a **template-bound, pay-first, text-only** invitation editor.

Companion files

File	Use
Engagement Sample 1 Timeline and Animation Details.docx	Layer positions, fonts, Canva animation names
Sample Engagement Invitation 1.mp4	Golden visual QA (founder sign-off)
FOUNDER_QA_CHECKLIST.md	Frame-by-frame acceptance tests
RESEARCH.md	Why we chose this stack (Canva research, rejected alternatives)

1. What you are building

1.1 Product in one sentence

After payment, the customer opens `/studio/:templateId`, watches a **9:16 looping video**, taps **text on the video** to edit names/dates/venue inline, previews **real animated text** synced to the timeline, then **exports an MP4** that matches the design reference.

1.2 What it is NOT

- Not a Canva integration (no Canva API; designers use Canva only as reference).
- Not a drag-and-drop layout tool (users cannot move, resize, recolour, or change fonts).
- Not a browser-side final render (export always runs on the server).
- Not a PDF animator in MVP (PDF path is static text only, separate product).

1.3 MVP scope

- **One template:** Engagement Sample 1.
- **Three animation types** in the engine (launch blocker until all match reference MP4).
- **Pay before studio:** no watermark trial editor.

2. High-level architecture

[illegible]

Golden rule: Preview and export must use the **same** animation-engine package. The browser uses DOM + CSS; the worker uses canvas. Never duplicate animation math in two places.

3. Editor user experience (build exactly this)

3.1 Screen layout

Region	Content
Main area	Nearly full viewport: 9:16 video letterboxed on desktop/mobile
Bottom (minimal)	Play, Pause, timeline scrubber , Done , Export
No primary sidebar	Do not use a form list as the main UI

3.2 Playback

- Video **loops** continuously while in the studio.

- User may **scrub** the timeline at any time.
- Use `video.currentTime` converted to **integer milliseconds** as the single time source for the animation engine.
- While **playing**, run `requestAnimationFrame` and update layer styles each frame.
- While **paused**, still update layers when the user scrubs.

3.3 Click-to-edit flow

1. User **clicks** on visible text (or its hit box).
2. Video **auto-pauses**.
3. That layer becomes **contenteditable** inline (same position as the animated text).
4. **All animation freezes** on every layer until the user finishes editing (hold static visible state).
5. Typing updates the overlay **in real time** (no debounce for preview).
6. User taps **Done** or **outside** the layer to deselect.
7. Animation resumes according to current `timeMs`.

3.4 Visual affordances

State	UI
Paused	Faint boxes around all editable layers in the current time window
Selected layer	Solid brand-colour border around that layer
Multiple layers under cursor	Small picker menu : “Which field?”

3.5 Hit testing

- Layer `x`, `y`, `width`, `height` live in **template canvas space** (from `definition_json`).
- Scale to screen: `scale = min(viewportW/canvasW, viewportH/canvasH)`; center letterboxed video.
- **z-index**: highest `zIndex` wins.
- **No hit slop**: tap target = exact box (critical on mobile).
- **Time gate**: layer is interactive only when `timeMs` is within that layer’s visibility window.

- **Baked intro:** for Engagement Sample 1, **no hit targets** for `timeMs < 6000` (page 1 is in the background video only).

3.6 Mobile

- When the keyboard opens, **scroll the video up** so the active field stays visible.
- Do not enlarge tap targets beyond the defined box.

3.7 Accessibility (WCAG 2.1 AA)

- **Tab** cycles through editable layers in `zIndex` order with a visible **focus ring**.
- Video controls need accessible names.
- Respect `prefers-reduced-motion` (reduce or disable motion in preview).

3.8 Persistence and navigation

- **Auto-save draft** to API (debounced) after payment.
- **Warn** on browser leave if unsaved local changes exist.
- Re-editing text after pay is **free**; only **downloads** consume quota.

4. Frontend module structure

Suggested location: `frontend/src/features/studio/`

Module	Responsibility
<code>StudioPage</code>	Route guard (entitlement), fetch template version + <code>definition_json</code> , load 1080p video URL
<code>StudioViewport</code>	Letterbox 9:16; compute scale/offset from canvas size
<code>StudioVideo</code>	<code><video></code> element, loop, <code>timeupdate</code> , seek
<code>LayerOverlay</code>	Positioned container for all layers
<code>EditableLayer</code>	One layer: hit area + text + selection state + <code>contenteditable</code>

<code>OverlapPicker</code>	Multi-hit resolution UI
<code>TransportBar</code>	Play / pause / scrub / Done / Export
<code>ExportModal</code>	Quality preset, estimated size, remaining downloads
<code>useStudioPlayback</code>	<code>timeMS</code> , <code>isPlaying</code> , <code>pause()</code> , seek handlers
<code>useStudioDraft</code>	Debounced save of <code>text_values</code> map to API

4.1 Route guard

- `/studio/:templateId` returns **403 or redirect** if user has no entitlement for that template version.
- Load **1080p proxy** URL from `definition_json` for preview.
- Do not load 4K in the browser (bandwidth and memory).

4.2 Text values model

Store user input as a map:

```
text_values: { [layerId: string]: string }
```

- Enforce `maxChars` and `maxLines` per layer in the UI (block further input).
- Reject emoji (strip or block on input).
- **Empty string** → layer omitted in preview and export.

5. Template data (`definition_json`)

Each published template version has **one JSON document** in `template_versions.definition_json`. The studio and worker must read **only** this document (plus user `text_values`).

5.1 Top-level fields

Field	Type	Description
-------	------	-------------

<code>canvasWidth</code>	number	Design canvas width (e.g. 1080)
<code>canvasHeight</code>	number	Design canvas height (e.g. 1920)
<code>durationMs</code>	number	Total video length (e.g. 24000)
<code>fps</code>	number	Export frame rate (60 for MVP)
<code>bakedIntroEndMs</code>	number	No editable layers before this time (6000 for Sample 1)
<code>backgroundVideoUrl1080</code>	string	Proxy for studio preview
<code>backgroundVideoUrl4K</code>	string	Master for 4K export
<code>fonts</code>	array	Font file references (storage paths)
<code>textLayers</code>	array	All editable layers

5.2 Text layer object

Field	Description
<code>id</code>	Stable key for <code>text_values</code>
<code>label</code>	Admin/debug label (e.g. "Partner 1 Name")
<code>x, y, width, height</code>	Pixels in canvas space (from Canva screenshots in design doc)
<code>fontFamily, fontSize, fontWeight, color</code>	Typography
<code>textAlign</code>	<code>left</code> / <code>center</code> / <code>right</code>
<code>maxChars, maxLines</code>	Validation
<code>zIndex</code>	Hit test stacking

<code>visibleFromMs, visibleToMs</code>	Optional hard window (in addition to animation phases)
<code>enter</code>	Enter animation block
<code>static</code>	Hold fully visible
<code>exit</code>	Exit animation block

5.3 Animation block shape

Each of `enter` and `exit`:

Field	Description
<code>type</code>	Engine slug: <code>fade_writing_element</code> , <code>skate_writing_rtl</code> , <code>burst_writing_enter</code>
<code>startMs</code>	Start time relative to video timeline (absolute ms from 0)
<code>durationMs</code>	Length of enter/exit phase
<code>params</code>	Type-specific JSON (easing, intensity, direction — from design calibration)

`static`:

Field	Description
<code>startMs</code>	First frame fully visible
<code>endMs</code>	Last frame fully visible before exit

5.4 Phase rules (engine must enforce)

1. **Before `enter.startMs`:** opacity = 0, **zero visible glyphs**, no hit target.
2. **Enter:** run enter handler from `startMs` for `durationMs`.
3. **Static:** full text visible between `static.startMs` and `static.endMs`.
4. **Exit:** run exit handler (may be a **different** type than enter).
5. **Stagger:** each layer has its own `enter.startMs` — no global auto-stagger in code.

6. **Empty user text:** skip layer entirely in preview and export.

5.5 Calibrating Canva “% speed” to milliseconds

Canva’s animation speed slider (15%, 50%, etc.) is **not** milliseconds. Process:

1. Design exports reference MP4 (Sample Engagement Invitation 1.mp4).
2. Engineer steps through frames at enter/exit boundaries.
3. Set `durationMs` in JSON until preview matches reference.
4. Document final ms values in Admin / JSON for that template.

6. Animation engine (`packages/animation-engine`)

6.1 Package purpose

Pure TypeScript library — **no React, no DOM, no node-canvas**. Imported by:

- Studio preview (maps output → CSS).
- Render worker (maps output → canvas draw calls).

6.2 Public API (minimum)

```
getLayerVisibility(layer, textValue, tMs) → boolean getLayerState(layer, textValue, tMs) → { opacity: number // 0..1 translateX: number // px in canvas space translateY: number scale: number rotate: number // degrees if needed visibleGraphemeCount: number // for writing-style reveals }
```

- If `textValue` is empty → always invisible.
- If user is editing this layer (pass flag from studio) → return **frozen** static state at pause time.

6.3 Grapheme segmentation

Writing animations reveal text **one grapheme cluster at a time**:

- Use `Intl.Segmenter` (or a shared polyfill) with locale `hi` / `en` as appropriate.
- **Never** use `string.length` or raw UTF-16 code units for Hindi.

6.4 Time and frames

- All internal times: **integer milliseconds**.
- Frame index for export: `n = floor(tMs * fps / 1000)`.
- Use the **same formula** in browser when sampling for tests.

6.5 Handler registry

```
handlers: Record<string, AnimationHandler> type AnimationHandler = (ctx: { layer,
text, tMs, phase: 'enter' | 'static' | 'exit', params }) => LayerState
```

Register exactly **three** handlers for v1 launch.

7. The three v1 animation handlers

7.1 `fade_writing_element`

Canva reference: Fade + Writing Style: Element (15% or 50% speed on slide bar).

Enter behaviour:

- Opacity eases from 0 → 1 over enter duration (params control curve).
- `visibleGraphemeCount` increases from 0 → full grapheme count over the same or linked duration (writing reveal).

Exit behaviour:

- Reverse: graphemes hide, opacity → 0 (when design specifies enter+exit).

Used on: Partner names, large names (pages 2–3).

7.2 `skate_writing rtl`

Canva reference: Skate, direction RTL, Writing Element (~50% speed).

Enter behaviour:

- Horizontal translation: text moves **right-to-left** into final position while graphemes reveal.
- Combine `translateX` + `visibleGraphemeCount` + opacity as needed.

Exit behaviour:

- Mirror enter for layers that exit (parent names on page 3).

Used on: Parent name fields.

7.3 `burst_writing_enter`

Canva reference: Burst + Writing (~50% intensity); **On enter only** in Canva.

Enter behaviour:

- Per-grapheme “pop” (brief scale overshoot + opacity) as each character appears.
- params may include burst intensity, stagger between glyphs.

Exit behaviour:

- **None** for MVP date/venue fields — layer stays visible through static window, then disappears at cut or hard hide per JSON (no burst exit).

Used on: Month, day, date, time, year, venue (page 4).

7.4 Launch gate

Do not ship Engagement Sample 1 to paying users until side-by-side frame comparison with Sample Engagement Invitation 1.mp4 passes at the checkpoints in `FOUNDER_QA_CHECKLIST.md`.

8. Browser preview implementation

8.1 Layer rendering

For each `textLayer`:

1. Call `getLayerState(layer, text_values[layer.id], timeMs)`.
2. If not visible, `pointer-events: none` and `opacity: 0`.
3. Apply to a positioned `div`:
 - `left/top/width/height` scaled to viewport.
 - `transform: translate(...) scale(...) rotate(...)`.
 - `opacity`.
4. Render only the first `visibleGraphemeCount` graphemes of the user string (substring by grapheme index).
5. Match `fontFamily`, `fontSize`, `fontWeight`, `color`, `textAlign` from JSON.

8.2 Fonts in browser

- Serve exact font files from Supabase Storage / `public/fonts`.
- `@font-face` with `font-display: block` for studio route.
- Load fonts before first preview frame (`await document.fonts.ready`).

8.3 Editing mode

When `activeLayerId` is set:

- Pass `isEditing: true` into engine **for all layers** (freeze entire composition).
- Show `contentEditable` on active layer only.
- Optionally hide faint boxes on non-active layers or keep them per design.

8.4 Coordinate scaling

```
displayX = offsetX + layer.x * scale displayY = offsetY + layer.y * scale displayW =  
layer.width * scale displayH = layer.height * scale
```

Use the same `scale` for transform translations output by the engine.

9. Server render pipeline

9.1 When a render runs

- User clicks **Export**, selects quality preset (4K / 2K / 1080p / WhatsApp-optimized).
- API checks entitlement + remaining download quota.
- If cached output exists for same `text_values` hash + quality → return immediately.
- Else enqueue `render_jobs` → worker.

9.2 Worker steps

1. Load `definition_json`, user `text_values`, quality preset (output width/height, bitrate).
2. Download **background MP4** (4K or 1080p depending on preset) and font files.
3. Register fonts with `node-canvas` (`registerFont`); set `ctx.lang` for Indic layers.
4. Spawn FFmpeg:
 - Input 1: raw RGB24 frames on stdin (`-f rawvideo -pix_fmt rgb24 -s wxH -r 60 -i -`).

- Input 2: background video (for audio copy and optional background frame source).
- For `n = 0 ... totalFrames - 1`:
 - `tMs = floor(n * 1000 / fps)`.
 - Clear canvas; draw background frame at `tMs` (decode from MP4 or pre-extracted strip).
 - For each layer with non-empty text, `getLayerState` → draw text on canvas with transform.
 - Convert canvas to **RGB24** (strip alpha); write to FFmpeg stdin with backpressure (`drain` event).
 - End stdin; wait for FFmpeg exit code 0.
 - Upload output to Supabase Storage; update `render_jobs` + `render_outputs`.

9.3 Audio

- **Copy audio** from background MP4 (`-c:a copy`) when possible.
- Do not re-encode unless required for container compatibility.

9.4 Quality presets

Preset	Typical output	Notes
4K	2160×3840 (9:16)	Use <code>backgroundVideoUrl4K</code> ; scale layer coordinates
2K	1440×2560	Scale from template canvas
1080p	1080×1920	May match template canvas 1:1
WhatsApp-optimized	720×1280 or similar	Lower bitrate cap; show estimated size in UI

Scale rule: All layer `x/y/width/height` are stored in template canvas space. At render time:

```
renderScale = outputWidth / definition_json.canvasWidth
```

Apply to positions and to engine `translate` values.

9.5 Performance

- 60 fps × 24 s ≈ **1440 frames** (Engagement Sample 1).
- Budget **worker timeout ~300s** for 4K.

- Cache shaped glyph layouts per (font, text) string where possible.
- Plan Redis queue concurrency limits on Render.

9.6 Failure handling

- On worker error: job failed, **do not** increment downloads_used.
- Show user WhatsApp support link with order id + job id.
- Admin can upload manual fulfillment file.

10. Engagement Sample 1 — concrete requirements

Page	Time (s)	Editable content	Animation
1	0–6	None (baked in video)	—
2	6–12	Partner 1, Partner 2	fade_writing_element 15%, enter+exit; Partner 2 same timing window as Partner 1
3	12–18	Large names, parents ×2	Names: fade 15%; Parents: skate_writing_rtl 50%, enter+exit
4	18–24	Month, day, date, time, year, venue	burst_writing_enter 50%, enter only

Fonts: Lucien Schoenschrift CAT, Lora (bold where specified), Futura (parent names).

Example positions (transcribe all 16 from design doc):

Layer	W×H	X, Y
Partner 1 (page 2)	372.3 × 120.3	353.8, 444.5
Partner 2 (page 2)	292.8 × 118.4	393.6, 629.2

Partner 1 large (page 3)	735.6 × 144	172.2, 1175.3
Month (page 4)	244.6 × 58.8	187.2, 926.2

11. Preview vs export parity checklist

#	Requirement
1	Same animation-engine package version in frontend and worker builds
2	Same font files and <code>fontSize</code> in CSS and canvas
3	Same <code>tMs</code> → frame formula
4	Same grapheme segmentation library
5	Empty layers skipped in both paths
6	RGB24 to FFmpeg (no accidental alpha premultiply difference)
7	Automated tests: fixed <code>tMs</code> → expected <code>opacity</code> , <code>visibleGraphemeCount</code> , <code>translateX</code>
8	Manual: frame grab comparison vs <code>Sample Engagement Invitation 1.mp4</code>

12. Export and download UX

- **Export modal:** show preset, **estimated file size**, downloads remaining.
- **Progress:** poll `render_jobs.status`; show % in UI.
- **Email** when render completes.
- **Download:** authenticated stream only (no public CDN URL).
- **Every download** decrements quota by 1, even if file was cached.

13. Edge cases

Case	Behaviour
Click during baked intro (0–6s)	Ignore or show tooltip “Not editable”
Multiple layers under click	Overlap picker
User clears field	Layer hidden; no animation
Render in progress	User may leave; poll continues
Failed render	No quota charge; support path
Long Hindi string	Wrap within box; grapheme reveal still applies
prefers-reduced-motion	Reduce/disable motion in CSS preview

14. Build order (recommended)

1. **packages/animation-engine** + unit tests for 3 types at fixed tms.
2. **Seed definition_json** for Engagement Sample 1 + upload 1080p/4K backgrounds + fonts.
3. **Studio UI** without export (preview only) — validate against reference MP4.
4. **API** drafts + entitlement guard.
5. **Render worker** + one quality preset; then add all presets.
6. **Razorpay** gate before studio.
7. **Export modal** + job polling + download.
8. **Admin** JSON import/export for template versions.
9. **Founder QA** checklist sign-off.

15. Definition of done

- ☐ Pay-first gate on /studio
- ☐ Click-to-edit with freeze-on-edit
- ☐ Page 1 not editable (0–6s)

- [] Three animations match reference MP4
- [] Hindi + English sample text render correctly in preview and export
- [] 4K / 2K / 1080p / WhatsApp presets work
- [] Preview and export pass parity tests
- [] Failed render does not consume quota

End of implementation guide.